

# LLD Learning Series - Day 2

Topics: Inheritance, Polymorphism and UML Diagrams

---

## Inheritance

Real-world objects often exist in parent-child relationships. Inheritance mimics this in code: a child class automatically gets all the attributes and behaviours of its parent, and can then add its own on top.

### Real-Life Example: Car Hierarchy

Parent Class: Car (Generic)

Attributes: brand, model, isEngineOn, currentSpeed

Behaviours: startEngine(), stopEngine(), accelerate(), brake()

Child Classes:

ManualCar (inherits Car)

Specific attribute: currentGear

Specific behaviour: shiftGear()

ElectricCar (inherits Car)

Specific attribute: batteryPercentage

Specific behaviour: chargeBattery()

### C++ Syntax

```
class ManualCar : public Car { ... };  
class ElectricCar : public Car { ... };
```

### Access Specifiers in Inheritance

| Specifier | Public members become | Protected members become | Private members |
|-----------|-----------------------|--------------------------|-----------------|
| public    | public                | protected                | Never inherited |
| protected | protected             | protected                | Never inherited |
| private   | private               | private                  | Never inherited |

---

## Polymorphism

Derived from "poly" (many) and "morph" (forms). One stimulus produces different responses based on the object or situation.

### Real-life scenarios:

Different animals (Duck, Human, Tiger) all have a run() behaviour but each performs it differently.

The same human runs differently based on context: tired vs being chased.

## Types of Polymorphism

| Type                 | When Resolved | Achieved Via       |
|----------------------|---------------|--------------------|
| Static Polymorphism  | Compile time  | Method Overloading |
| Dynamic Polymorphism | Runtime       | Method Overriding  |

### Static Polymorphism: Method Overloading

Same method name, different parameter lists. The compiler decides which version to call at compile time based on the arguments passed.

```
class ManualCar {  
    void accelerate(); // no parameter  
    void accelerate(int speed); // with parameter  
};
```

Rules for overloading:

- Method name must be the same.

- Parameters must differ in number or type.

- Return type alone cannot distinguish overloaded methods.

### Dynamic Polymorphism: Method Overriding

The same method signature is redefined in child classes. In C++ this requires virtual functions. The correct version is resolved at runtime based on the actual object type.

```
class Car {  
    virtual void accelerate() = 0; // Abstract  
};  
class ManualCar : public Car {  
    void accelerate() override; // Manual logic  
};  
class ElectricCar : public Car {  
    void accelerate() override; // Electric logic  
};
```

## All Four OOP Pillars Working Together

The car hierarchy demonstrates all four pillars at once:

- Abstraction: Car exposes startEngine(), accelerate() etc. without revealing how they work.

- Encapsulation: Internal state like currentGear and batteryPercentage is private/protected.

- Inheritance: ManualCar and ElectricCar inherit common behaviour from Car.

- Polymorphism: accelerate() behaves differently for each child class at runtime.

---

## UML Diagrams

UML (Unified Modeling Language) diagrams visually represent the structure and behaviour of an application. Instead of writing lengthy paragraphs, UML lets you communicate classes, objects, entities, and their interactions in a clear diagrammatic form.

### Types of UML Diagrams

| Category   | What it shows                              | Count   |
|------------|--------------------------------------------|---------|
| Structural | Static structure of the system             | 7 types |
| Behavioral | Interaction between components and objects | 7 types |

Of all 14 types, only two matter for LLD and interviews:

Class Diagram (Structural): represents classes and their static relationships.

Sequence Diagram (Behavioral): represents the order of messages between objects at runtime.

### Class Diagram

A class diagram shows the system's classes, their attributes (variables), methods (behaviours), and the associations between them.

### How a Class is Represented

A class is drawn as a rectangle divided into three compartments:

Top: Class name (write 'abstract' above the name for abstract classes).

Middle: Attributes listed as accessModifier name : dataType

Bottom: Methods listed as accessModifier methodName() : returnType

### Access Modifier Symbols

| Symbol | Modifier  | Accessibility                           |
|--------|-----------|-----------------------------------------|
| +      | public    | Accessible everywhere                   |
| #      | protected | Accessible within class and subclasses  |
| -      | private   | Accessible only within the class itself |

### Example: Car Class

| Part       | Content                                                                     |
|------------|-----------------------------------------------------------------------------|
| Attributes | +brand: String +model: String +engineCC: Integer                            |
| Methods    | +startEngine(): void +stopEngine(): void +accelerate(): void +brake(): void |

## Associations Between Classes

Associations show how classes relate to each other. There are four main types:

| Type               | Relationship                            | UML Notation                                               | Example                  |
|--------------------|-----------------------------------------|------------------------------------------------------------|--------------------------|
| Inheritance        | Is-a                                    | Solid line with closed hollow arrowhead pointing to parent | ManualCar is-a Car       |
| Simple Association | Has-a (loose)                           | Open arrow between classes                                 | Arjun has-a House        |
| Aggregation        | Has-a, parts can exist independently    | Line with hollow diamond at the whole end                  | Room has-a Sofa, Bed     |
| Composition        | Has-a, parts cannot exist without whole | Line with filled diamond at the whole end                  | Chair has-a Wheels, Seat |

Key distinction: Aggregation is a weaker relationship where parts can live independently. Composition implies the parts cannot exist without the whole.

## Sequence Diagram

A sequence diagram captures the dynamic behaviour of a system. It shows the sequence of interactions between objects during runtime: who sends a message to whom, and in what order.

### Key Components

| Component      | What it represents                            | UML Representation                 |
|----------------|-----------------------------------------------|------------------------------------|
| Objects        | Instances taking part in the interaction      | Rectangle labeled with object name |
| Lifeline       | Duration of an object's existence             | Dashed vertical line below object  |
| Activation Bar | Period when object is active/executing        | Thin rectangle on the lifeline     |
| Messages       | Communication or method calls between objects | Arrows between lifelines           |

### Types of Messages

| Message Type | Description                                 | UML Representation                |
|--------------|---------------------------------------------|-----------------------------------|
| Synchronous  | Caller waits for response before continuing | Solid arrow + dotted return arrow |
| Asynchronous | Caller sends and continues without waiting  | Open arrow                        |

|                |                                      |                                               |
|----------------|--------------------------------------|-----------------------------------------------|
| <b>Create</b>  | Creates a new object                 | Special message that initiates a new lifeline |
| <b>Destroy</b> | Destroys an object                   | Marks the end of a lifeline                   |
| <b>Lost</b>    | Message sent but never received      | Arrow ending with a circle                    |
| <b>Found</b>   | Message received from unknown source | Incoming arrow from no known sender           |

### Additional Constructs

| Construct   | Meaning                                       |
|-------------|-----------------------------------------------|
| <b>alt</b>  | if-else conditional flow                      |
| <b>opt</b>  | if condition only (optional block)            |
| <b>loop</b> | Repetitive operations: for loop or while loop |

### Example: ATM Withdrawal Flow

Objects involved: User, ATM, Transaction, Account, Cash Dispenser

1. User sends withdraw(amount, accNo) to ATM.
2. ATM creates a Transaction object.
3. Transaction sends checkAmt(amount) to Account.
4. Account returns true (balance verified).
5. ATM sends withdrawCash(amt) to Cash Dispenser.
6. Cash Dispenser returns the amount to ATM.
7. ATM returns the amount to User.

This flow shows object activation, synchronous messaging, and a return path. An alt block could wrap step 3 to handle insufficient balance as an else case.

---

## My Takeaways from Today

1. Inheritance is not just code reuse. It is modelling a real-world is-a relationship. Always ask: does this child truly IS-A parent, or does it just share some behaviour?
2. Polymorphism is what makes systems flexible. The same method call behaves differently depending on the object, without the caller needing to know which type it is.
3. Method overloading is resolved at compile time (static). Method overriding is resolved at runtime (dynamic). These are fundamentally different mechanisms.
4. In a class diagram, the three-compartment rectangle (name / attributes / methods) is the universal language. Access modifier symbols (+, #, -) must be accurate.

5. Aggregation vs Composition comes down to one question: can the part exist without the whole? If no, it is composition. If yes, it is aggregation.
6. Sequence diagrams are about time and order. Reading one top to bottom tells you exactly what happens, in what sequence, and who is responsible at each step.
7. The ATM example is a perfect mental model for sequence diagrams. Every real-world transaction can be mapped this way.